

**COURSE NUMBER:** CP1895

**COURSE TITLE:** Advanced Python Programming

**COURSE DESCRIPTION:**

In this course, students will learn advanced Python programming concepts and features, expanding upon the fundamental skills acquired in the Introduction to Programming with Python. Core concepts include a variety of structures such as dictionaries, stacks, queues, sets, trees, and algorithmic complexities and the application of object-oriented design principles to build software using the appropriate data structures. Content will focus on the applied use of Python to solve modern-day problems utilizing one or more popular frameworks.

**PREREQUISITES:** CP1855 – Introduction to Programming with Python  
or  
CP1856 – Programming with Python  
or  
CP1850 – Procedural Programming  
or  
CP1035 – Programming Foundations

**CO-REQUISITES:** None

**CARNEGIE UNIT:** Three (3)

**CREDIT VALUE:** Four (4)

**COURSE HOURS PER WEEK:** Three (3)

**LAB HOURS PER WEEK:** Three (3)

**REQUIRED TEXT:**

Murach, J. (2021). *Murach's Python Programming*. (2<sup>nd</sup> Edition) Mike Murach & Associates.  
ISBN: 9781943872749, eText ISBN: 9781943872756

**LEARNING RESOURCES:**

[Python Institute](#)

**MAJOR TOPICS:**

1.0 Data Structures and Algorithm Complexity

- 2.0 Classes and Objects
- 3.0 Object Oriented Programming
- 4.0 Emerging Trends in Applied Python

## **LEARNING OBJECTIVES:**

### **1.0 Data Structures and Algorithm Complexity**

- 1.1 Differentiate:
  - 1.1.1 Dictionaries
  - 1.1.2 Stacks
  - 1.1.3 Queues
  - 1.1.4 Hash tables
  - 1.1.5 Sets
- 1.2 Use dictionaries in python programs to manipulate key-value pairs
- 1.3 Implement a program using hash tables
- 1.4 Discuss the structure of a tree
  - 1.4.1 Show how binary trees are used for storing data
  - 1.4.2 Discuss the types of tree traversal such as “depth first” search and “breadth first” search
- 1.5 Explain algorithmic complexity
  - 1.5.1 Describe what is computational complexity
  - 1.5.2 Introduce asymptotic notation
  - 1.5.3 Discuss the complexity classes such as (but not limited to):
    - 1.5.3.1 Constant complexity
    - 1.5.3.2 Logarithmic complexity
    - 1.5.3.3 Linear complexity
    - 1.5.3.4 Polynomial complexity

### **2.0 Classes and Objects**

- 2.1 Describe the relationship between a class and an object
- 2.2 Code the constructor for a class that has attributes and methods
- 2.3 Distinguish between public and private attributes
- 2.4 Create objects by importing classes
- 2.5 Access the attributes of objects
- 2.6 Call methods on objects
- 2.7 Use object composition to combine simple objects into complex data structures
- 2.8 Use encapsulation to hide the data attributes of an object

### **3.0 Object Oriented Programming**

- 3.1 Explain python inheritance
- 3.2 Explain how to override a method in a superclass while defining a subclass

- 3.3 Describe polymorphism
- 3.4 Describe the use of the “isinstance()” method when working with objects
  - 3.4.1 Describe the use of object classes and methods of the object class including “\_\_str\_\_()”, “\_\_iter\_\_()”, “\_\_eq\_\_,” and “\_\_next\_\_()”
- 3.5 Use a subclass that inherits a superclass and overrides methods of the superclass
- 3.6 Use a class that overrides one or more methods of an object class

## 4.0 Emerging Trends in Applied Python

- 4.1 Use current or emerging industry standards and frameworks in python such as Flask, Django, NumPy, Panda, Matplotlib or Maching Learning
  - 4.1.1 Web Development using Flask and Django
    - 4.1.1.1 Introduce the fundamental concepts to build web applications using the Python Flask framework
      - 4.1.1.1.1 Discuss the advantages of using Flask
      - 4.1.1.1.2 Install Flask and build a basic Flask web application
      - 4.1.1.1.3 Demonstrate how templating works in Flask
    - 4.1.1.2 Introduce creating web applications using the Django framework
      - 4.1.1.2.1 Discuss the advantages of using for creating web applications
      - 4.1.1.2.2 Build a web app with multiple views using the Django framework
  - 4.1.2 Implement data science concepts utilizing Numpy, Panda and Matplotlib
    - 4.1.2.1 Use NumPy to work with N-dimensional array, Indexing and slicing of arrays, Indexing using integer arrays, transposing an array, universal functions, data processing using arrays
    - 4.1.2.2 Introduce Pandas’ high-level data structures and manipulation tools for data analysis
    - 4.1.2.3 Use Matplotlib to generate line graphs, histograms, and pie charts
  - 4.1.3 Discuss Machine Learning
    - 4.1.3.1 Discuss supervised vs unsupervised learning
    - 4.1.3.2 Discuss regression vs classification
    - 4.1.3.3 Using a machine learning framework, create a simple program demonstrating a framework such as an image classifier

## EVALUATION:

|              |     |
|--------------|-----|
| Assignments: | 20% |
| Project:     | 20% |
| Tests (2):   | 30% |

Final Exam: 30%

*This course is supplementary eligible.*

**DATE DEVELOPED:** December 2019

**DATE REVIEWED:**

**REVISION NUMBER:** 5

**DATE REVISED:** May 2025

Effective PeopleSoft Date: September 2025

*Note to instructor: Check PIRS to ensure this outline is the most current version.*